

AI Capability Assessment

Practical & AI-Allowed · Take-Home + Live Defense

TAKE-HOME	WINDOW	LIVE DEFENSE	AI TOOLS	SUBMIT VIA
~3-4 hrs work	3 days	30 minutes	Allowed	GitHub URL

CANDIDATE DETAILS — PLEASE FILL IN

FULL NAME	EMAIL
POSITION / TRACK (SE OR AI ENGINEER)	DATE RECEIVED
GITHUB REPOSITORY URL (YOUR SUBMISSION)	

■ HOW THIS WORKS — READ FIRST

This is not a memory test. We want to see how well you **use AI** — your judgment, your skepticism, and your ability to turn an AI's draft into something correct and trustworthy.

- **AI tools are allowed and encouraged.** Use whatever you normally use (Claude, Copilot, Cursor, ChatGPT, etc.).
- **Show your seams.** Keep a **PROCESS_LOG** in your repo: the key prompts you used, the raw AI output, and exactly what you changed and why.
- **Commit as you go.** We read your **commit history** — steady, meaningful commits beat one giant final dump.
- **Shortlisted candidates defend their work live** (30 min): reproduce part of it, make a small change on the spot, and explain any line. Work you can't explain counts for nothing.

Pick the track closest to the role you applied for. If unsure, do Track A.

■ ENVIRONMENT SETUP & REPOSITORY STRUCTURE

Set up a clean, runnable project on GitHub. We must be able to clone it and run Part 1 by following your README.

- Create a repository named `ctmedtech-assessment-<your-name>` . Put your **full name** at the top of the README.
- If the repo is **private**, grant access to `[github-username/email]` .
- Pin your environment: include `requirements.txt` / `package.json` , a `.env.example` (never commit real secrets/keys), and exact run commands.
- Include a `/skills` folder with at least one `SKILL.md` — a short, reusable "recipe" describing an AI workflow you used (e.g., how you prompt for test generation, or your RAG-eval procedure). We want to see that you can **encode and reuse** your AI workflows, not just one-off prompts.

```
ctmedtech-assessment-<your-name>/
├─ README.md           # your name, how to run, where each answer lives
├─ PROCESS_LOG.md      # prompts + AI outputs + your edits & reasoning
├─ requirements.txt     # or package.json – pinned deps
├─ .env.example        # config keys, no real secrets
├─ skills/
│   └─ SKILL.md        # a reusable AI workflow you actually used
├─ src/                # Part 1 – your build
│   └─ tests/          # your tests (incl. the edge case)
├─ part2_debug.md      # Part 2 – defects, fixes, how verified
├─ part3_hallucination.md # Part 3 – your analysis + sources
└─ part4_reflection.md  # Part 4 – your answers
```

Tidy structure and a clear README are part of the score — they show how you'd actually work on a team.

■ TIMELINE

When	What
Day 0 — [date]	You receive this assessment.
By end of Day 1	Create the repo, set up the environment, and push your first commit (a "hello, it runs" baseline). This confirms you've started.
By Day 3, 23:59 — [deadline]	Finish Parts 1–4, push your final commits, and reply with your GitHub URL .
Day 4–5 — [scheduled]	30-minute live defense for shortlisted candidates (we'll book a slot with you).

Need more time for a good reason? Tell us early — we value honesty over a rushed submission.

Part 1 — Build With AI (and show the seams)

~90 MIN

Build a small, working solution for **one** track below. Using AI is expected. What we examine is the **hard constraint** — the part naive AI output usually gets wrong.

Track A — Software Engineer. Build a small HTTP service with one endpoint `GET /search?q=&page=` over a provided (or mocked) dataset of ~10,000 records. Hard requirements: correct **pagination**, **case- and accent-insensitive** search (Vietnamese text included), a sensible **response time** on the full dataset, and graceful handling of **malformed input**. Include at least 3 tests, one covering an edge case AI is likely to miss.

Track B — AI Knowledge Engineer. Build a minimal **RAG** over the 5 short documents we provide. Hard requirements: answers must **cite** the source passage; the system must **refuse / say "not in the documents"** when the answer isn't present (no hallucinating); and it must handle a question whose answer is **split across two documents**. Include a tiny eval: 5 questions (3 answerable, 2 that must be refused).

Deliver: working code under `/src` + your `PROCESS_LOG` + a 5-line note **"Where AI helped, and where it was wrong."**

Part 2 — Debug the AI's Code

~45 MIN

The function below was generated by an AI assistant. It runs and looks reasonable, but contains **three distinct defects**: one **security** issue, one **resource / robustness** issue, and one **performance** issue. Find all three. For each: explain **what** is wrong, **how you found it**, fix it, and state **how you verified** the fix. Put answers in `part2_debug.md`.

```
import sqlite3

def get_users_by_ids(db_path, user_ids):
    conn = sqlite3.connect(db_path)
    cur = conn.cursor()
    results = []
    for uid in user_ids:
        query = "SELECT id, name, email FROM users WHERE id = '%s'" % uid
        cur.execute(query)
        row = cur.fetchone()
        if row:
            results.append({"id": row[0], "name": row[1], "email": row[2]})
    return results
```

Language is Python, but the concepts apply broadly. Don't just paste the whole thing back to an AI and copy the reply — in the live defense you'll be asked to explain your detection process for each defect.

Part 3 — Catch the Hallucination

~30 MIN

Below are four confident statements produced by an AI. For **each**: say whether it is **correct or incorrect**; if incorrect, give the correction; and state **how you verified** it (official docs, source, a quick experiment). Not all are wrong, and at least one references something that does not exist. Put answers in `part3_hallucination.md`.

S1. "In Python, `list.sort(reverse=True, key=len, stable=False)` sorts a list in place; pass `stable=False` when you want a faster unstable sort."

S2. "In PostgreSQL with the `pgvector` extension, the `<=>` operator computes cosine distance between two vectors."

S3. "Fundus photography directly measures intraocular pressure (IOP), so it can replace tonometry for glaucoma screening."

S4. "Setting `temperature=0` on an LLM API guarantees the model returns exactly the same output every time for the same prompt."

Part 4 — Judgment & Reflection

~30 MIN

Answer in your own words in `part4_reflection.md`. Be specific and personal — generic, AI-sounding answers score low, and you'll be asked to expand on these live.

- 1 Describe a **real** situation where an AI gave you a wrong or misleading answer. How did you notice, and what did you do?
- 2 In your Part 1 solution, name **one** place where you deliberately did **not** use AI — and explain why.
- 3 How do you keep AI-generated code **maintainable and trustworthy** when shipping to production with a team?
- 4 You're working with **patient / medical data**. What are your personal rules for using AI tools responsibly in that context?

■ SUBMISSION & LIVE DEFENSE

- Push everything to GitHub and **reply with your repository URL** before the Day 3 deadline.
- Make sure it **runs from a clean clone** by following your README. If we can't run it, we can't score Part 1.
- Shortlisted candidates join a **30-minute live session**: reproduce a slice of Part 1, implement a **small change we ask on the spot**, and walk us through your Part 2 and Part 3 reasoning. You may use AI during the live session too — we're watching **how** you use it.

■ HOW YOU'RE SCORED

Dimension	What we look for	Weight
Verification & skepticism	Catching the planted defects and hallucinations; verifying with real sources.	25%
Solution quality	Part 1 correctness, edge cases, and the hard constraint actually handled.	20%
AI collaboration	Prompt quality, knowing when to trust vs. rewrite, clear <code>PROCESS_LOG</code> , a real <code>SKILL.md</code> .	20%
Engineering hygiene	Repo structure, README, commit history, reproducible setup, no committed secrets.	15%
Judgment & live defense	Part 4 depth, responsible use, and explaining/extending your work under questioning.	20%